

Payments and credits

What a diver owes, what they have paid, what the shop owes them back, and what happens to each when something is cancelled.

The app moves no money. There is no payment processor. Bank transfers, cash and card payments happen off-app and are recorded by hand. Account credit is the one exception — it is purely internal, and the only "payment" the app creates itself.

The model

Four append-only sources. Nothing is ever edited in place: a refund is a new `payments` row, a discount a new `booking_amendments` row.

Source	Table	Means
Charge snapshot	<code>bookings.details</code> (<code>total</code> , <code>deposit</code> , <code>charges</code>)	What was quoted. Frozen at booking.
Adjustments	<code>booking_amendments</code>	Signed rows: + surcharge, - discount.
Money received	<code>payments</code>	What actually arrived.
Money owed back	<code>credits</code>	What the shop owes the diver.

```
owed      = details.total + Σ booking_amendments.amount
paid      = Σ payments.amount signed by status
credit    = Σ open credits.amount tied to THIS booking
balance   = owed - paid - credit                      (bookingBalance)
depositDue = max(0, min(details.deposit, owed) - paid)
```

`bookingBalance()` returns `due`, `settled`, or `credit`. The `credit` state covers both an awarded credit larger than what is owed and a plain overpayment — an overpayment is money owed back, so there is no separate "overpaid" concept.

The deposit is clamped to owed. `details.deposit` is frozen and amendments never reduce it, so a discount can push it above the remaining balance. Without the clamp a diver who paid their discounted total in full would still be shown a deposit due, and their booking would never auto-confirm.

Card/PayPal surcharge (`business.cardSurchargePercent`) applies only to what actually goes on the card *now*: the whole subtotal when paying in full, the deposit only when paying deposit-only. It is folded into the `total` / `deposit` snapshots, not a separate ledger line.

The payments ledger

Status	Counts toward paid	Meaning
<code>pending</code>	no	Recorded intent; not yet confirmed.
<code>paid</code>	+amount	Money received.
<code>refunded</code>	-amount	Money sent back. Its own row.
<code>voided</code>	no	"Never really happened." Kept for the audit trail.

`netPaid()` / `netPaidByBooking()` (`src/lib/payments.ts`) and `public.booking_net_paid()` are the only implementations of that signing rule. Every surface that shows a paid figure must use one. When they disagree, a partly refunded booking looks more paid to the RPC than to the screen and credit applications silently under-apply.

RLS: **admins only** write `payments`. A diver's only path to the ledger is `apply_credit_to_booking`.

Promotion rule: a pending booking becomes `confirmed` as soon as the deposit (clamped to `owed`) is covered. `recordPayment`, `apply_credit_to_booking` and `record_group_payment` each implement it and must stay identical; `voidPayment` mirrors it in reverse.

Group payments. `bookings.payer_id` marks a lead booker, per booking so an admin can revert one diver. `record_group_payment` distributes one lump across the group — deposits first, then balances, oldest first — inserting one ordinary `payments` row per booking, so all the math above is unchanged. It nets each booking's own open credit out first, or cash would be collected against money already offsetting the balance.

The credits ledger

`public.credits` — money the shop owes a diver. `status` is `open` (spendable, still owed) or `settled` (paid back, or spent).

`source` records where the row came from:

source	Written by
<code>manual</code>	An admin, on the Users page
<code>event_cancellation</code>	<code>issueCancellationCredits</code> — the shop called off an event
<code>booking_cancellation_return</code>	The return-on-cancel trigger, or an admin resolving the holding queue
<code>carry_forward</code>	The unspent remainder of a partly-consumed credit

Only the two middle values (`RETURN_SOURCES` in `src/lib/credits.ts`) mean *this booking's money has been given back*, and only they suppress a further automatic refund. A goodwill award or a carry-forward remainder tied to the same booking is unrelated money and must never block one.

Account credit

The figure on the Profile and Payments pages is **not** the sum of open credit rows:

```
account credit =  $\Sigma$  open credits NOT tied to an active booking
                +  $\Sigma$  over active bookings:  $\max(0, \text{paid} + \text{tiedCredit} - \text{owed})$ 
```

General credit **plus every overpayment**. A credit tied to a *cancelled* booking counts as general credit — that is how a cancellation credit lands back in the spendable pool. Bookings someone else pays for (`payer_id`) are dropped from both terms: that money is the lead's.

`diverCreditBalance()` is the only definition. Note the **spendable pool** is different and smaller: `openCreditBalance()`, actual credit rows only, because the RPC consumes rows. An overpayment shows as account credit but cannot be moved by the RPC — that takes an admin.

Spending credit

One RPC: `apply_credit_to_booking(p_booking_id, p_amount)`, SECURITY DEFINER, because divers cannot write `credits` or `payments`. Callable by the diver for their own booking, a parent for their child's, an admin for anyone's. The credit spent is always the **booking owner's**.

1. Refuses a **cancelled** booking — its frozen total is not money owed, and burning credit into it would destroy it.
2. `due = owed - netPaid - credit already tied to this booking`. Self-tied credit already offsets this booking on screen.
3. Pool = the diver's open credit **excluding** rows tied to this booking.
4. Applies `least(requested, due, pool)`; returns it. Never raises for "not enough credit".
5. Consumes rows oldest-first. The row straddling the boundary is settled in full and its remainder re-issued as `carry_forward`.
6. Inserts one `account_credit` payment in the credit's currency.
7. Applies the promotion rule.

Callers must refetch. `plannedCreditApplication()` replays this logic client-side so the "use my credit" button promises what it will deliver.

At registration the form applies credit *after* the booking lands (best-effort — the booking already succeeded).

Cancellation: what happens to the money

There are four cancellations and they do not do the same thing.

What happened	Booking rows	Money effect
Shop cancels the event	Bookings unchanged; <code>events.cancelled_at</code> set	Automatic — every non-cancelled registrant gets an open credit worth their full net paid
Refund request approved	Booking → <code>cancelled</code>	Manual
Admin cancels one booking	Booking → <code>cancelled</code>	Manual
Diver cancels their own booking	Booking → <code>cancelled</code>	Manual

In the three manual cases the `bookings_return_account_credit_on_cancel` trigger does return any **account credit** spent on the booking — that is the one method whose refund is purely internal. Bank transfer, cash and card moved off-app, so only a person can move them back.

The consequence, and the thing most likely to confuse an admin:

When a diver who paid by bank transfer or cash cancels, **nothing happens to their money automatically**. No credit appears. The booking shows a settled balance. The shop still holds the cash.

Exactly two endings are correct:

- **Give it back** → record a `refunded` payment row. Net paid goes to zero.

- **Keep it as store credit** → issue an open credit stamped `booking_cancellation_return`, so no later automatic issuer pays the same money out again.

Doing neither hides the money. Doing **both** pays the diver twice.

Finding it: the holding queue

`/admin/refunds` carries a second list, "**Cancelled bookings still holding money**" — every cancelled booking with positive net paid and no credit saying the money came back. It is the only surface that can see this money: `bookingBalance` short-circuits a cancelled booking to "settled", `diverCreditBalance` drops cancelled bookings, and the refund-request queue lists only *non*-cancelled bookings.

Each row offers the two endings above and disappears once one is taken. An empty list means every cancellation is accounted for. Approving a refund request moves a booking straight into this queue — correct, since the approval cancelled the booking but the cash has not moved.

Event cancellation, in detail

`cancelEventAndFollowUp()` stamps `cancelled_at`, notifies every registrant (fire-and-forget), then `issueCancellationCredits()` issues each non-cancelled booking an open credit worth its **net paid**. Bookings with nothing paid get none. Idempotent per booking on `RETURN_SOURCES`, so cancel → restore → cancel never double-issues. Crediting runs after the cancel commits, so a failure cannot un-cancel the event — the admin is told to issue them by hand.

Who may cancel what

A diver may self-cancel only a `pending` booking with nothing paid and no refund request outstanding — `canSelfCancel()`. Otherwise both the Bookings page and the calendar modal offer **Request refund** instead. This is a product rule, not a security boundary: the DB guard `bookings_guard_diver_status` lets a diver cancel any of their own bookings (it only stops self-confirming and re-homing). So the rule must be applied at **every** diver-facing cancel control, through the shared helper.

Cancellation policies (`cancellation_policies`, attached per event) are text the diver acknowledges at registration. Nothing computes a forfeiture from them; an admin enacts whatever they say by hand.

Why a cancelled booking reads "settled"

`bookingBalance(..., { cancelled: true })` short-circuits to zero. A cancelled booking keeps its frozen total, so netting it normally would show the diver owing the rest of an event that will not happen, and would double-count any cancellation credit.

"Settled" here means "nothing further is owed on this event", not "the money has been dealt with."

Reporting

Surface	Netting
<code>/admin/audits</code>	Reconciles with the same <code>bookingBalance()</code> as the app. Its totals block is deliberately gross.
<code>/admin/accounting export</code>	paid +, refunded −, voided shown but excluded.
<code>/admin/dashboard</code>	Same signed rule, cash only.

Surface	Netting
Revenue by staff	Base price × confirmed heads — deliberately not <code>netPaid</code> . See admin.md .

Account credit is not revenue. An `account_credit` row settles a booking without anything arriving — the cash arrived earlier, on the booking that generated the credit. Counting it books the same money twice. `isExternalPayment()` is the predicate every cash-revenue aggregation applies; credit applied is reported *beside* cash (its own dashboard KPI, its own export column and summary line). Balance math is the opposite and counts it in full — this is a reporting predicate, not a change to `netPaid`.

Reminders fire at 21/14/7/3/1 days out, skipping cancelled bookings: `depositDue > 0` → deposit message; else `balanceDue > 0` → balance message; else nothing. See [push-notifications.md](#).

Known gaps

1. **The confirmation PDF's credit line is a prediction.** Credit is applied after the booking lands, but the PDF renders inside `create-registration` before that. When the apply fails the diver is told on the success screen, but the emailed PDF has already gone.
2. **`credits.currency` / `payments.currency` default to `TWD`.** Every writer names a currency explicitly now, except `record_group_payment` recording the *first* money on a booking, which has nothing to inherit from (`events` carries no currency column). A fork on another currency should change the column defaults in its own baseline.
3. **Restoring a cancelled event leaves its credits issued** — deliberate, but a cancel/restore cycle leaves divers holding credit for an event that is running again.

Invariants

1. **Never edit money rows in place.** New row, always.
2. **Sum paid through `netPaid` / `netPaidByBooking` / `booking_net_paid`.**
3. **`owed` is `details.total` plus amendments** — in TypeScript and SQL.
4. **Clamp the deposit to `owed`** wherever you compare against it.
5. **The three promotion rules stay identical**, and `voidPayment` mirrors them.
6. **A booking's own credit is never spendable against that booking.**
7. **A cancelled booking has no live balance** and refuses credit.
8. **`diverCreditBalance` is the only definition of account credit**, and always excludes lead-covered bookings.
9. **Guards belong in the RPC or trigger, not only the UI.** Where a rule is a product rule rather than a security boundary (`canSelfCancel`), apply it at every control that offers the action.
10. **Every automatic credit records its `source`.** A new issuer means a new value, not `manual`.
11. **Cash-revenue sums apply `isExternalPayment`.** Balance math does not.
12. **Constraints, triggers and RLS policies get integration tests** — `tests/integration/{credits,apply-credit-to-booking,balance-consistency,return-account-credit-on-cancel,gaurd-diver-booking-status,lead-payer}` and `tests/scenario/event-cancellation.test.ts`.